UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIDAD DE APRENDIZAJE

COMPONENTES DE LÓGICA DE NEGOCIO

DOCENTE

M.G.T. MIGUEL ÁNGEL GÓMEZ MARROQUÍN

ACTIVIDAD

INFORME DE ARQUITECTURA

EMPRESA ASIGNADA: RUTANORTE LOGISTICS

SEMESTRE: AGOSTO – DICIEMBRE 2026

GRUPO: 002 **DÍA:** JUEVES **HORA:** N1, 3 **SALÓN:** 3-307

MATRÍCULA	ESTUDIANTE	CARRERA
2049234	BASURTO RIVERA RAMSÉS TENOSH	ITS
2050864	CERDA HERRERA JUAN KRISTOF	ITS
2066053	ESPARZA TORRES CARLOS ARTURO	ITS

FECHA DE ENTREGA: 15 DE OCTUBRE DE 2026

ALCANCE

En este documento, se halla el diseño arquitectónico de la capa de lógica de negocio del sistema RutaNorte Logistics, el patrón de lógica de negocio elegido con base en 5 casos de uso, el mapa de dominio con las entidades, agregados, objetos de valor, etc., el Service Layer, el Repository, Unit of Work, CQRS, la revisión con los principios SOLID y los tres ADR.

CASOS DE USO

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Solicitar y confirmar cotización	Cliente/ Ventas	La cotización tiene una vigencia de 5 días hábiles, se puede editar la orden antes de aceptar, calcular la tarifa.	Cotización generada, se pasa a *confirmada* si el cliente acepta dentro de los 5 días hábiles, se pasa a *vencida* de lo contrario.
CU-02	Generar orden de servicio y asignar unidad	Operaciones	Operaciones genera la orden de servicio y asigna un operador, una unidad, una ruta, un horario.	La orden pasa a fase *Unidad Asignada*, con recursos registrados.
CU-03	Registrar entrega con evidencia	Operador	Operador toma evidencia, foto de mercancía entregada y firma del destinatario.	La orden pasa a fase *Entregada* con evidencias guardadas en el mismo folio.
CU-04	Cancelar o devolver una orden según la fase	Cliente/ Operaciones	El Cliente solicite la cancelación/devolución de la orden, con penalización según la fase de la orden.	Dependiendo de la fase de la orden, se cancela sin costo, se cancela con penalización, o se devuelve con penalización más cargo de regreso.
CU-05	Reasignar unidad por avería en tránsito	Operaciones	Gestión de excepciones conocidas, y notificar al cliente.	Orden actualizada con un nuevo operador/unidad notificando al cliente, o cancelada en caso de no encontrar reemplazo.

PATRÓN DE LÓGICA DE NEGOCIO

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Para cada caso de uso se programaría un procedimiento aislado, ocasionando que se repitan scripts	Centralizaría por tabla, pero para este negocio dependería de varias tablas, por ejemplo en un momento se pueden cruzar la tabla de orden, operaciones y operador, y empieza a fallar en su propia lógica	Puede evitar duplicación, por ejemplo la orden de servicio sabe si puede cancelarse o devolverse según el estado en el que se encuentra	N-06 que habla sobre órdenes de cancelación/devolución, N-10 que trata sobre las cargas de alto valor, N-12 que es sobre las tarifas de caso no estándar
Cambios esperados	El hacer un mínimo cambio, provocaría que todos los scripts se cambien, si aplican	Los cambios en una tabla no afectan otras, pero se duplican reglas que cruzan tablas	El haber un cambio en el objeto dueño, cambia todos los casos de uso que lo invoquen	N-06 habla también del cálculo de penalización, que depende de múltiples condiciones que es probable que el negocio quiera ajustar sin tocar código de presentación.
Interacción entre conceptos	Cada uno de los scripts está aislado, por lo que hay poca interacción entre conceptos	Modela tablas, no relaciones. No existiría una tabla de orden de cambio sin una de orden de servicio	Por ejemplo, una orden de servicio puede controlar sus subelementos (cambios, evidencias, estados) como una unidad consistente	N-05, la trazabilidad de las órdenes que fueron modificadas. N-06 sobre órdenes canceladas/devueltas, N-03 sobre registro de evidencias completadas
Estados e invariantes	Puede no manejar muy bien las invariantes, ocasionando que dos o más scripts se interpongan	Las invariantes pueden no vivir naturalmente, se convierten en reglas externas que consulte entre tablas	El propio objeto (como una orden de servicio) puede rechazar una transición inválida protegiendo el invariante donde vive el dato	BRD, el ciclo de trabajo a través de las distintas fases del servicio con transiciones condicionadas
Criterio adicional	En una cancelación/ devolución, cada script tiene que volver a hacer el cálculo de penalización por fase	La tabla de Órdenes tendría que guardar lógica de cálculo embebida, mezclando datos con comportamiento de forma rígida	La orden de servicio centraliza el cálculo una sola vez, consultado por cualquier caso de uso que lo necesite	N-06, las penalizaciones según el estado de la orden en que se encuentre

El patrón de lógica de negocio denominado como Transaction Script, organiza la lógica de un negocio en diversos scripts, en procedimientos individuales, donde cada uno de estos maneja un solo flujo completo, o sea un solo procedimiento. Este patrón de lógica de negocio es útil cuando existen pocas reglas y el flujo cambia poco; es útil cuando se trata de un negocio, por decirlo así, pequeño.

El otro patrón que se comparó fue el Table Module, el cual funciona en base a tablas. Organiza la lógica con una clase por tabla de la base de datos, y una única instancia de una clase contiene los distintos procedimientos que actuarán sobre los datos. Este patrón es útil cuando predominan reportes.

Por último, se comparó el patrón Domain Model, este tiene un enfoque donde la lógica de negocio se organiza dentro de una red estructurada de objetos que contienen tanto datos como comportamiento, y este interactúa activamente aplicando las reglas del negocio.

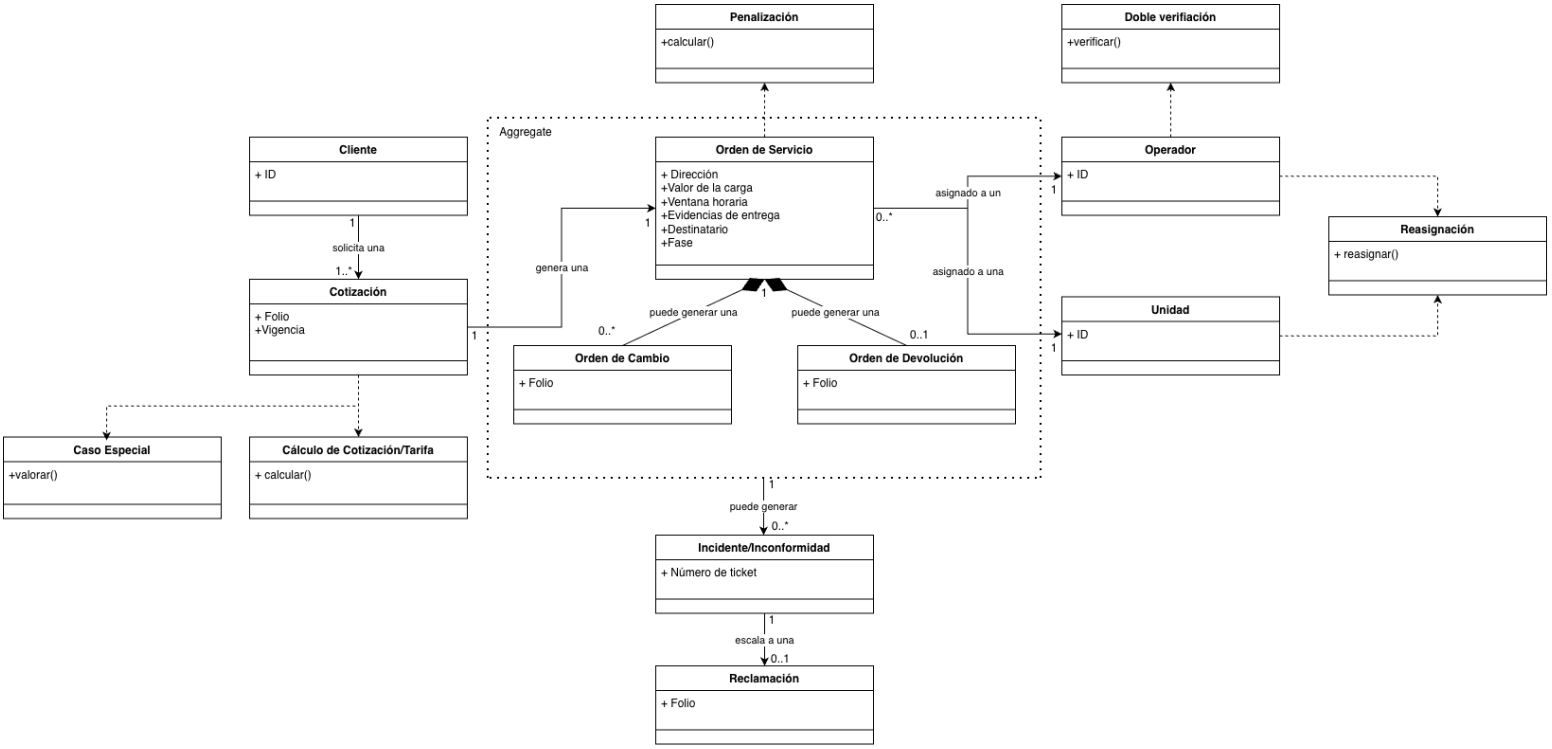
RutaNorte Logistics funciona a base de un ciclo de vida de 8 estados aproximadamente, donde pueden haber cambios y excepciones documentadas y reglas de negocio que dependen del estado del objeto. Es por ello que el patrón más adecuado para RutaNorte Logistics es el patrón Domain Model, ya que este permite que el conocimiento del negocio (qué transiciones de estado son aceptables, cómo se calcula una penalización, cuándo se necesita una doble verificación) viva dentro de los objetos del dominio (Orden de Servicio, Cotización, Cliente), en lugar de estar disperso en varios procedimientos.

Si bien el Domain Model es más complejo que los otros dos patrones, este modelo puede llegar a ser mucho más mantenible. Sin embargo, este requiere un proceso inicial más complejo, al identificar los Aggregates, las Entidades, los Value Objects, etc.

Transaction Script no es una buena opción porque RutaNorte maneja más de 15 reglas de negocio interdependientes (cotización, cancelación, excepciones, alto valor) que se duplicarían entre scripts; y Table Module tampoco, esto debido a que varias de esas reglas (como por ejemplo: N-05, N-07) cruzan varias tablas (Orden, Cliente, Operador) y rompen el diseño centrado en una sola tabla.

MAPA DE DOMINIO

A continuación, se presenta el mapa de dominio relacionando los elementos del negocio a una categoría:



Categoría	Elemento del negocio
Entidades	Orden de Servicio, Operador, Cliente, Cotización, Unidad, Incidente/Inconformidad, Reclamación
Value Objects	Dirección, Valor de la carga, Ventana horaria, Evidencias de entrega, Destinatario
Aggregates	[Orden de Servicio (Raíz), Orden de Cambio, Orden de Devolución]
Domain Service	Doble verificación, Penalización (cálculo), Caso Especial, Cálculo de Cotización/Tarifa, Reasignación
Domain Events	OrdenDeServicioConfirmada, MercanciaEntregada, UnidadAveriada, OrdenDeCambioRegistrada, OrdenDeDevolucionGenerada, IncidenteRegistrado, ReclamacionEscalada, ReasignacionRealizada

Ficha del Aggregate de Orden de Servicio:

Categoría	Descripción
Raíz	Orden de Servicio (con un folio único, y una fase en la que se encuentra)
Responsabilidades	Al aceptarse una cotización, generarse una Orden de Servicio; avanzar de fase; generar una Orden de Cambio cuando aplique; generar una Orden de Devolución cuando aplique; registrar evidencias de entrega; aceptar o rechazar una cancelación según su fase
Invariantes	● Ninguna Orden de Cambio o de Devolución debe existir sin su Orden de Servicio ● Un cambio de fase se rechaza dentro de la orden ● Una carga de alto valor no pasa a “entregada” sin una doble verificación ● El cálculo de una penalización sólo es válida si la orden está “cancelada” o en “devolución”
Eventos posibles	OrdenDeServicioConfirmada, MercanciaEntregada, OrdenDeCambioRegistrada, OrdenDeDevolucion

Una decisión que se tomó fue el tener un solo Aggregate, esto debido a que el único presente agrupa varios objetos con una sola raíz (Orden de Cambio y Orden de Devolución), además de tener invariantes que se cruzan entre ellos. Si bien, las otras Entidades también son un Aggregate (Operador, Cliente, Cotización, Unidad, Incidente/Inconformidad, Reclamación), porque cada uno es raíz de sí mismo y porque nadie los contiene, son por así decirlo, triviales, no se consideraron para las fichas de Aggregates; cada uno es su propia raíz, sin objetos internos que proteger.

SERVICE LAYER

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
AppSer_Cotizacion	SolicitarYConfirmaCotizacion (clienteID, destino, origen, mercancia, peso, dimensiones, fecha) CU-01	Datos de la solicitud, clienteID	ClienteRepository, CotizacionRepository Domain Services: Cálculo de Cotización/Tarifa, Caso Especial	CotizacionDTO (folio, estado)	Vigencia de 5 días hábiles, se puede editar antes de aceptar (N-08, N-12)
AppSer_OrdenDeServicio	GenerarOrdenYAsignarUnidad (cotizacionID, unidadID, operadorID, ruta, horario) CU-02	cotizacionID, unidad, operador, ruta, horario	OrdenDeServicioRepository, CotizacionRepository, UnidadRepository, OperadorRepository	OrdenDeServicioDTO (folio, fase)	Cotización debe estar confirmada, un operador no puede tener otra orden en tránsito (N-07)
AppSer_OrdenDeServicio	RegistrarEntrega (ordenID, evidencia) CU-03	ordenID, evidencias de Entrega	OrdenDeServicioRepository Domain Service: Doble verificación	OrdenDeServicioDTO (actualizada)	Doble verificación obligatoria si la carga es de alto valor (N-10)
AppSer_OrdenDeServicio	CancelarOrdenODevolver (ordenID, motivo, autorizadoPorOperaciones) CU-04	ordenID, motivo, autorizadoPorOperaciones	OrdenDeServicioRepository Domain Service: Penalización	CancelacionDevolucionDTO	Con unidad asignada requiere autorización de Operaciones; penalización según la fase, genera devolución si ya está en ruta (N-06, US-008)
AppSer_Incidente	ReasignarPorAveria (ordenID, nuevaUnidadID, nuevoOperadorID) CU-05	ordenID, nueva unidad y/u operador	OrdenDeServicioRepository, UnidadRepository, OperadorRepository, IncidenteRepository Domain Service: Reasignación	OrdenDeServicioDTO (actualizada), notifica al cliente	Si no hay reemplazo en 4 horas, se cancela la orden y se notifica al cliente (N-09)

PERSISTENCIA Y TRANSACCIONES

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
CotizacionRepository	AppSer_Cotizacion necesita crear una nueva cotización con su número de folio para modificarla o confirmarla y guardar su estado	guardar (cotizacion: Cotizacion): void obtenerPorFolio (folio: string): Cotizacion cotizacion.esVigente (): Boolean	Entidad: Cotizacion
ClienteRepository	AppSer_Cotizacion necesita verificar que el cliente exista, y si sí, y si es recurrente para aplicar tarifas preferenciales	obtenerPorID (clienteID: string): Cliente esRecurrente (clienteID: string): Booleano	Entidad: Cliente
OrdenDeServicioRepository	AppSer_OrdenDeServicio necesita guardar la orden recién generada (CU-02), recuperarla por folio para registrar la entrega (CU-03) o cancelarla o devolverla (CU-04), y saber si un operador ya tiene una orden en tránsito	guardar (orden: OrdenDeServicio): void obtenerPorFolio (folio: string): OrdenDeServicio obtenerEnTransitoDelOperador (operadorID: string): OrdenDeServicio[]	Aggregate: OrdenDeServicio (raíz)
UnidadRepository	AppSer_OrdenDeServicio necesita comprobar que la unidad existe y está disponible antes de asignarla (CU-02)	obtenerPorID (unidadID: string): Unidad obtenerDisponibles (): Unidad[] guardar (unidad: Unidad): void	Entidad: Unidad
OperadorRepository	AppSer_OrdenDeServicio necesita comprobar que el operador existe y está disponible para el servicio (CU-02, N-07)	obtenerPorID (operadorID: string): Operador obtenerDisponibles (): Operador[] guardar (operador: Operador): void	Entidad: Operador
IncidenteRepository	AppSer_Incidente necesita recuperar el incidente abierto de una orden en tránsito para reasignar unidad y/u operador (o cancelar la orden si pasan 4 horas), y guardar su estado al terminar.	guardar (incidente: Incidente): void obtenerPorID (incidenteID: string): Incidente obtenerAbiertoPorOrden (folio: string): Incidente	Entidad: Incidente

Implementación en infraestructura de OrdenDeServicioRepository, UnidadRepository y OperadorRepository: las interfaces se declaran en la capa de dominio y sus implementaciones (por ejemplo, SqlOrdenDeServicioRepository, SqlUnidadRepository y SqlOperadorRepository) viven en infraestructura. SqlOrdenDeServicioRepository guarda y reconstruye el Aggregate completo (orden, órdenes de cambio, orden de devolución y evidencias) en una sola operación, de modo que nunca se persiste una parte suelta. Los repositorios solo guardan y recuperan: saber si un operador puede tomar una orden no es un método del repositorio, esa regla se decide en el dominio. El código base incluye implementaciones en memoria de las mismas interfaces para las pruebas.

Unit of Work

SolicitarYConfirmarCotizacion: La transacción inicia al entrar al método AppSer_Cotizacion.SolicitarYConfirmaCotizacion(...). Incluye: crear (o recuperar y actualizar) el registro de Cotizacion en CotizacionRepository, validando contra ClienteRepository que el cliente exista. Si falla la validación del cliente, o si la cotización ya venció al momento de intentar confirmarla, se revierte la transacción completa, no se guarda ningún cambio parcial, y el cliente recibe un error indicando el motivo (cliente no encontrado, o cotización vencida).

GenerarOrdenYAsignarUnidad: La transacción inicia al entrar al método AppSer_OrdenDeServicio.GenerarOrdenYAsignarUnidad(...). Incluye: recuperar la Cotizacion desde CotizacionRepository y comprobar que esté confirmada y vigente, recuperar la Unidad y el Operador desde UnidadRepository y OperadorRepository, verificar la regla de disponibilidad del operador (solo puede tener una orden en tránsito), crear la Orden de Servicio, asignarle unidad, operador, ruta y horario (fase "Unidad asignada") y guardar la orden y la unidad. Si la cotización no está confirmada o ya venció, el operador ya tiene una orden en tránsito o la unidad no está disponible, se revierte la transacción completa, no se crea la orden ni se asigna nada, y se devuelve el motivo en términos del negocio. El evento OrdenDeServicioConfirmada se publica solo después del commit.

RegistrarEntrega: La transacción inicia al entrar al método AppSer_OrdenDeServicio.RegistrarEntrega(ordenID, evidencia). Incluye: recuperar la Orden de Servicio desde OrdenDeServicioRepository, validar con el Domain

Service Doble verificación que estén las evidencias obligatorias (foto y firma, y además identificación oficial si la carga es de alto valor), agregar las evidencias a la orden, pasarla a la fase "Entregada" y guardarla. Si la orden no está en el punto de entrega o falta evidencia, se revierte la transacción, la orden conserva su fase y no queda evidencia parcial. El evento MercanciaEntregada se publica solo después del commit.

CancelarOrdenODevolver: La transacción inicia al entrar al método AppSer_OrdenDeServicio.CancelarOrdenODevolver(...), que delega en CancelarOrdenODevolverHandler. Incluye: recuperar la Orden de Servicio, calcular con el Domain Service Penalización lo que corresponde según su fase (sin unidad asignada se cancela sin costo; con unidad asignada se cancela con penalización solo si Operaciones lo autoriza; en ruta no se cancela, se genera una Orden de Devolución con penalización y cargo de regreso) y guardar la orden completa. Si falta la autorización de Operaciones con unidad asignada, o si la orden ya no puede cancelarse (por ejemplo, ya fue entregada), se revierte la transacción, la orden queda como estaba y no se cobra una penalización parcial. Los eventos OrdenDeServicioCancelada u OrdenDeDevolucionGenerada se publican después del commit, y es entonces cuando se notifica al cliente.

COMMANDS Y QUERIES

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	ConfirmarCotizacion	cotizacionID	AppSer_Cotizacion.ConfirmarCotizacion(cotizacionID) Depende de CotizacionRepository	CotizacionDTO (folio, estado “Confirmada”)	Cambia el estado del sistema (de “Cotizada” a “Confirmada”), válido solo si la cotización sigue vigente dentro de los 5 días hábiles
Command	CancelarOrdenODevolver	ordenID, motivo, autorizadoPorOperaciones	CancelarOrdenODevolverHandler Depende de OrdenDeServicioRepository, Penalización y Unit of Work	CancelacionDevolucionDTO (folio, fase resultante, penalización aplicada, cargo de regreso)	Cambia el estado de la orden. La penalización por fase y la autorización de Operaciones con unidad asignada viven en el dominio, no en el handler (N-06, US-008)

Query	ConsultarCotizacion	folio	Lee directo de CotizacionRepository.obtenerPorFolio(folio)	CotizaciónVistaDTO (folio, estado, diasRestantesVigencia)	No cambia el estado, solo muestra información para que Ventas o el Cliente revisen antes de decidir
Query	ConsultarEstadoDeOrden	folio	ConsultarEstadoDeOrdenHandler Lee de un modelo de lectura propio (ModeloDeLecturaDeEstado), no del repositorio de escritura	EstadoDeOrdenVistaDTO (folio, faseActual, fechaHoraUltimoCambio, responsableDelCambio)	No cambia el estado. Atiende a Atención al Cliente y al Cliente sin cargar el Aggregate completo (N-02, US-002)

DTO de lectura

CotizacionVistaDTO: (folio: string, estado: string, diasRestantesVigencia: int)

Es distinto del CotizacionDTO de escritura, que solo trae (folio, estado). El de lectura agrega diasRestantesVigencia, un dato calculado útil para quien consulta, que no tiene sentido guardar en el modelo de escritura porque se calcula al momento de leer.

EstadoDeOrdenVistaDTO: (folio: string, faseActual: string, fechaHoraUltimoCambio: datetime, responsableDelCambio: string)

Es distinto del CancelacionDevolucionDTO de escritura (folio, fase resultante, penalización aplicada, cargo de regreso) y no reutiliza el Aggregate: solo muestra lo que necesita quien consulta, sin exponer evidencias ni reglas internas. Se actualiza con los eventos de dominio publicados después del commit. Leer la orden para registrar una entrega (CU-03) no se modela como Query: es una lectura interna del propio caso de uso.

REVISIÓN SOLID

- Hallazgo 1

Single Responsibility (S)

Evidencia: cada Domain Service tiene una sola razón para cambiar: Penalización calcula, Doble verificación valida las evidencias, Reasignación valida unidad y operador disponibles. En el ADR-003 las reglas del caso de avería se repartieron por objeto: el Incidente sabe si venció el plazo de 4 horas (plazo_vencido), la Orden de

Servicio protege sus cambios de fase (cancelar_por_falta_de_reemplazo, reasignar_recursos) y AppSer_Incidente solo coordina.
Impacto: si cambia la fórmula de penalización o el plazo de 4 horas, se modifica un solo objeto, y los Application Services no cambian.
Cambio realizado: durante la revisión se encontró una regla de negocio que había quedado en un Application Service y se movió al dominio. Además, la regla de disponibilidad del operador (N-07) se decide en el dominio y no en el repositorio.

- Hallazgo 2

Open/Closed (O)

Evidencia: la lógica de penalización vive aislada en Penalizacion.calcular(), y las reglas por fase están en una tabla de reglas, no en una cadena de if. En la parte de consultas, el EstadoDeOrdenVistaDTO se alimenta de un modelo de lectura propio (ADR-002).
Impacto: agregar una fase o una variante de cobro es agregar una entrada a la tabla, sin editar condicionales existentes. Agregar un campo de lectura (por ejemplo el tiempo estimado de llegada de US-019) no obliga a tocar el Aggregate.
Cambio realizado: se mantuvo la penalización como tabla de reglas en lugar de una cadena de if. Al comprobar US-008 se agregó la autorización de Operaciones para cancelar con unidad asignada como una columna más de la tabla, sin que creciera ninguna cadena de condiciones

- Hallazgo 3

Liskov (L)

Evidencia: las implementaciones en memoria del código base cumplen el mismo contrato que las interfaces de Repository, para que los casos de uso funcionen igual con cualquiera. El contrato de CotizacionRepository.obtenerPorFolio es devolver la Cotización, o None si el folio no existe. Es AppSer_Cotizacion quien convierte ese caso en un error de negocio (“no existe una cotización con folio X”).
Impacto: una implementación (en memoria o SQL) puede sustituir a otra sin cambiar el comportamiento esperado del Application Service.

Cambio realizado: se corrigió el contrato para que coincida con lo implementado, porque antes se había descrito como un error lanzado por el repositorio. Queda por confirmar con la implementación SQL real, y con los repositorios de Orden, Unidad, Operador e Incidente, que todos respeten el mismo comportamiento cuando no encuentran el folio.

- Hallazgo 4

Interface Segregation (I)

Evidencia: cada Repository expone solo lo que necesita su Aggregate o Entidad, y los Domain Services tienen interfaces de un solo método. La consulta ConsultarEstadoDeOrden no depende del repositorio de escritura: lee de ModeloDeLecturaDeEstado (ADR-002). En la tabla de Repository, CotizacionRepository listaba cotizacion.esVigente(), que es un comportamiento de la Entidad, no una operación de persistencia.
Impacto: quien consulta no depende de métodos de escritura ni de reglas del Aggregate, y los repositorios no cargan con comportamiento que no les corresponde.
Cambio realizado: se separó el modelo de lectura del repositorio de escritura para la consulta de estado de orden, y la vigencia (esVigente) se dejó en la entidad Cotización, no en el repositorio.

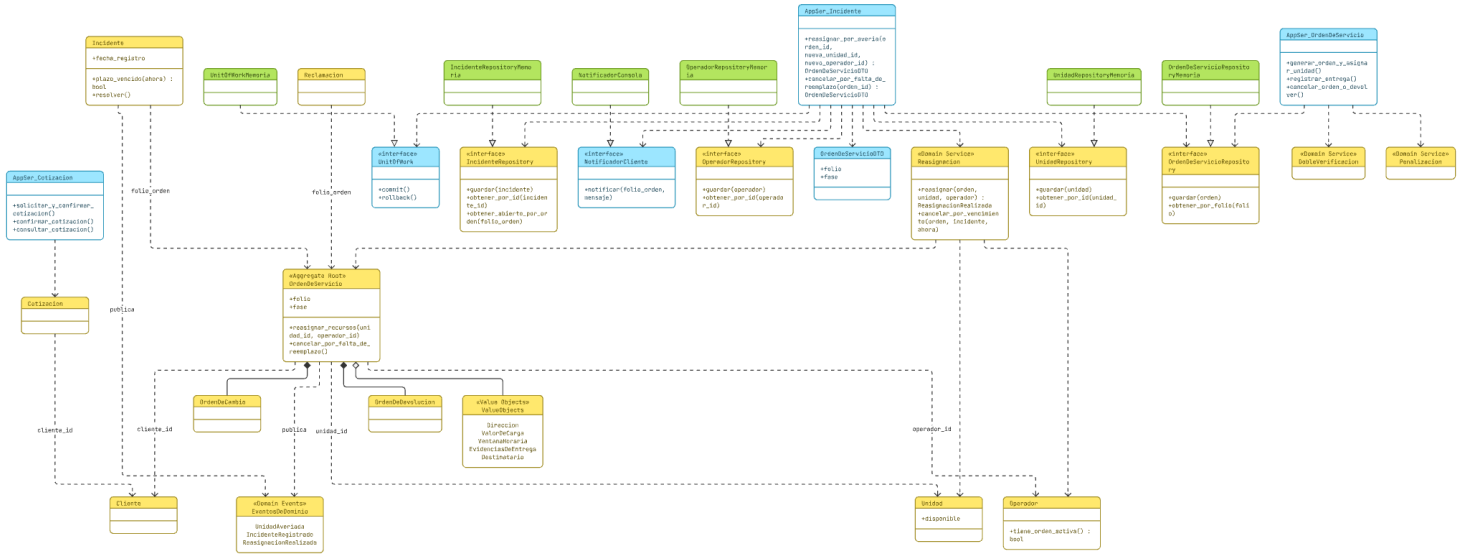
- Hallazgo 5

Dependency Inversion (D)

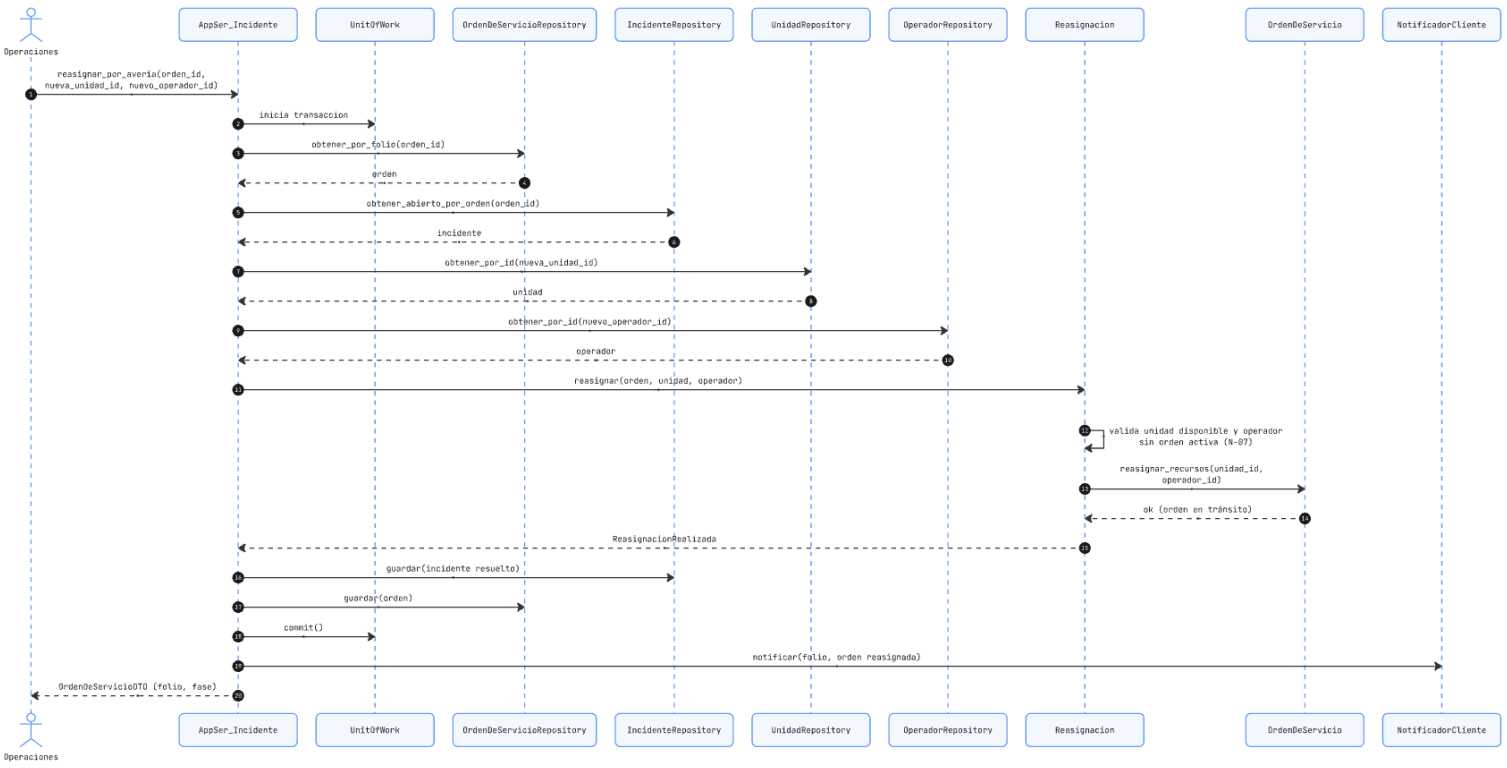
Evidencia: las interfaces de Repository se declaran en la capa de dominio y sus implementaciones (SqlOrdenDeServicioRepository, SqlUnidadRepository, SqlOperadorRepository) viven en infraestructura. Los Application Services (por ejemplo AppSer_Cotizacion) reciben el repositorio como interfaz abstracta, y el código base incluye implementaciones en memoria para las pruebas.
Impacto: se puede cambiar de base de datos, o probar sin base de datos, sin tocar el Application Service ni el dominio.

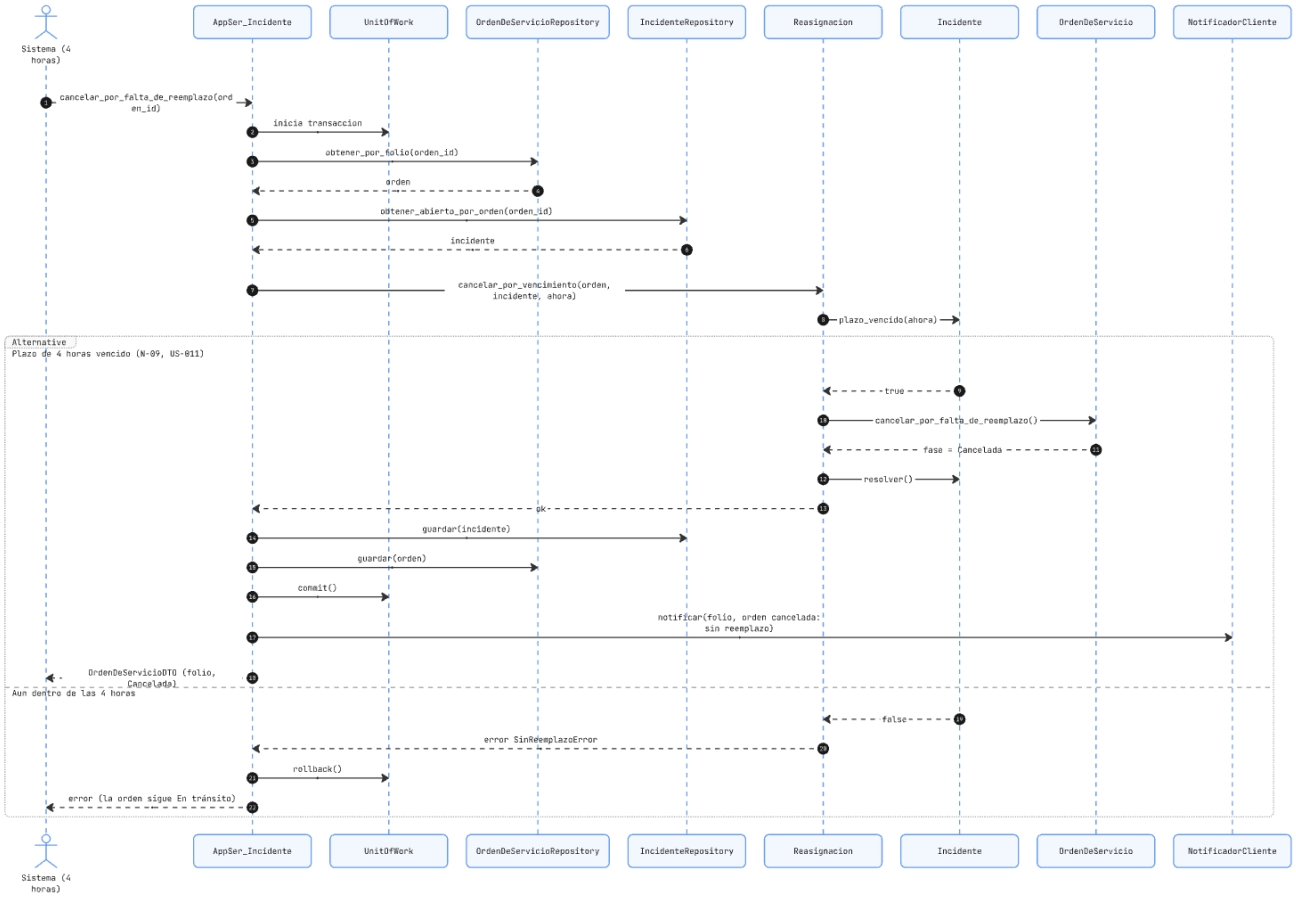
Cambio realizado: durante el diseño se movieron los métodos que mezclaban infraestructura con lógica de negocio, así que los repositorios quedaron solo con guardar y recuperar, y la dependencia apunta de infraestructura hacia el dominio.

DIAGRAMA DE CLASES



DIAGRAMAS DE SECUENCIA





ARCHITECTURE DECISION RECORDS

ADR - 001: Domain Model como patrón de lógica de negocio

Estado: Aceptada

Contexto

RutaNorte Logistics transporta mercancía entre Monterrey, Saltillo, Reynosa, San Luis Potosí y Querétaro. Necesita digitalizar y optimizar el seguimiento de una orden/pedido/servicio de principio a fin. En el que los operadores dejen de comunicarse por medios informales como WhatsApp, tener registro de las fases o estados en tiempo real y obtener una organización en cuanto a las evidencias presentadas.

Alternativas

- Transaction Script: organiza la lógica de un negocio en diversos scripts, en procedimientos individuales, donde cada uno de estos maneja un solo flujo completo, o sea un solo procedimiento.
- Table Module: funciona en base a tablas. Organiza la lógica con una clase por tabla de la base de datos, y una única instancia de una clase contiene los distintos procedimientos que actuarán sobre los datos.
- Domain Model (seleccionada): tiene un enfoque donde la lógica de negocio se organiza dentro de una red estructurada de objetos que contienen tanto datos como comportamiento, y este interactúa activamente aplicando las reglas del negocio.

Decisión

RutaNorte Logistics funciona a base de un ciclo de vida de 8 estados aproximadamente, donde pueden haber cambios y excepciones documentadas y reglas de negocio que dependen del estado del objeto. Es por ello que el patrón más adecuado para RutaNorte Logistics es el patrón Domain Model.

Consecuencias

Beneficios: un cambio en una regla de negocio (por ejemplo, el porcentaje de penalización por cancelación) se modifica en un solo lugar y también se cambia automáticamente en todos los casos de uso que lo invocan

Costos: este modelo es más complejo que los otros dos patrones, este modelo puede llegar a ser mucho más mantenible. Sin embargo, este requiere un proceso inicial más complejo, al identificar los Aggregates, las Entidades, los Value Objects, etc.

Riesgos: se necesita disciplina del equipo para no filtrar lógica de negocio hacia los Application Services (lo que rompería el principio de Service Layer) ni hacia la capa de infraestructura (lo que acoplaría el dominio a detalles de base de datos).

Trabajo posterior: implica diseñar interfaces de Repository y Unit of Work para separar la persistencia del dominio, y mantener cada Domain Service con una sola responsabilidad. También implica que cualquier nueva regla de negocio futura deberá evaluarse primero para decidir en qué objeto de dominio vive, en lugar de añadirse directamente a un Application Service.

Trazabilidad

N-06, N-08, N-09, N-10, N-12

ADR - 002: Consultar el estado de una orden desde un modelo de lectura separado del Aggregate

Estado: Propuesta

Contexto

Atención al Cliente y el Cliente necesitan conocer el estado de una orden sin depender de Operaciones (N-02, US-002, US-019). La consulta solo necesita folio, fase, fecha y hora del último cambio y quién lo hizo (US-002). La Orden de Servicio, en cambio, es un Aggregate con evidencias, órdenes de cambio, devolución y reglas de fase. Si la consulta cargara el Aggregate, mezclaría lectura y escritura. El volumen es moderado (supuesto del BRD), así que el rendimiento no es el criterio principal.

Alternativas

- Leer del OrdenDeServicioRepository y mapear al DTO: es simple y no duplica datos, pero carga el Aggregate completo para mostrar cuatro campos y ata la consulta al modelo de escritura.
- Vista SQL en infraestructura: es rápida, pero deja la consulta fuera de la aplicación, sin handler ni contrato, y acopla la lectura a la base de datos.
- Modelo de lectura propio actualizado con los Domain Events (seleccionada): aísla la lectura del dominio y permite un DTO pensado para quien consulta, pero obliga a mantener la proyección.

Decisión

ConsultarEstadoDeOrdenHandler lee de ModeloDeLecturaDeEstado, que se actualiza con los eventos publicados después del commit (OrdenDeServicioConfirmada, MercanciaEntregada, OrdenDeServicioCancelada y OrdenDeDevolucionGenerada). EstadoDeOrdenVistaDTO no reutiliza el modelo de escritura.

Consecuencias

Beneficios: la consulta no depende de las reglas del Aggregate, el DTO muestra solo lo que cada consumidor necesita y se pueden agregar campos de lectura sin tocar el dominio.
Costos: hay que mantener la proyección, y la lectura puede ir un instante detrás de la escritura porque se actualiza después del commit.
Riesgos: si un evento no se publica, la vista queda desactualizada. Se mitiga publicando solo después del commit y probando la proyección con cada evento.
Trabajo posterior: US-019 también pide el tiempo estimado de llegada y N-04 pide tiempos de inicio y fin por fase; el DTO actual no incluye ninguno de los dos.

Trazabilidad
N-02, N-04 (parcial), US-002, US-019

ADR-003: Incidente como Aggregate independiente de Orden de Servicio

Estado: Aceptada

Contexto
CU-05 (Reasignar unidad por avería en tránsito) obliga a registrar un incidente cuando una unidad se avería y a reasignar unidad y/u operador. Si no hay reemplazo en 4 horas, la orden se cancela y se notifica al cliente (N-09). En el mapa de dominio, Incidente/Inconformidad es una Entidad con identidad y ciclo de vida propio, mientras que el único Aggregate con ficha es Orden de Servicio, junto con Orden de Cambio y Orden de Devolución. Había que decidir si el Incidente forma parte de ese Aggregate o vive aparte.

Alternativas
Incidente dentro del Aggregate de Orden de Servicio. Todo se guarda junto y es más simple de transaccionar. Pero la orden crecería con algo que no protege invariantes suyos, y un incidente (o una reclamación posterior) obligaría a cargar y bloquear toda la orden.
Solo campos de avería en la Orden, sin entidad Incidente. Es lo más simple, pero se pierde el historial (varias averías en una misma orden) y contradice el mapa de dominio, donde Incidente es Entidad.

Incidente como Aggregate independiente, referenciado por folio_orden (elegida). Tiene su propio repositorio y su propio ciclo de vida.

Decisión

Incidente es su propio Aggregate, con raíz propia y IncidenteRepository. Solo conoce a la orden por su folio. Las reglas se reparten así: el Incidente sabe si venció el plazo de 4 horas (plazo_vencido), la Orden de Servicio protege sus cambios de fase (cancelar_por_falta_de_reemplazo, reasignar_recursos), y el Domain Service Reasignación valida la unidad disponible y que el operador no tenga otra orden activa (N-07). AppSer_Incidente solo coordina.

Consecuencias

Beneficios: la Orden no se infla con datos de averías. Los incidentes se consultan y se cierran sin tocar la orden. Se conserva el historial y el lenguaje del mapa de dominio.

Costos: un mismo caso de uso modifica dos Aggregates (Orden e Incidente), así que el Unit of Work de AppSer_Incidente debe incluir ambos cambios en una sola transacción. Si falla uno, se revierte todo.

Riesgos: acoplar demasiados Aggregates en una transacción. Si crece, conviene separar con eventos de dominio (UnidadAveriada, IncidenteRegistrado).

Trabajo posterior: validar con el BRD si la cancelación por falta de reemplazo lleva penalización, y definir quién dispara el vencimiento de las 4 horas (Operaciones o un proceso automático).

Trazabilidad

CU-05, N-07, N-09.

CONCLUSIONES

2049234 BASURTO RIVERA RAMSÉS TENOSH:

Como conclusión, la realización de cada *etapa* de esta actividad confirmó que el patrón adecuado para RutaNorte Logistics es Domain Model, esto debido a que las reglas de negocio son complejas, que pueden crecer y modificarse, haciendo que las entidades puedan tener un comportamiento propio.

Para la elaboración del Mapa de Dominio, la Service Layer y el Repository, no se trabajaron de una manera lineal, de una manera continua, esto porque mientras se avanzaba, se encontraban ciertos errores en las tomas de decisiones, que se fueron corrigiendo, que no concordaban con el BRD ni las User Stories, como lo fueron el identificar erróneamente ciertos elementos como Domain Services en lugar de Entidades, no poder identificar de manera correcta los Aggregates, métodos que combinaban infraestructura con lógica de negocio, (coordinación), etc. Fue el corroborar continuamente con el BRD y las User Stories para verificar la lógica y proceso en lugar de solo asumir, además de con ayuda de IA, fue que se pudieron corregir estas cosas.

Estas actividades fueron reforzadoras para aprender a separar las capas como dominio, infraestructura y aplicación, con el fin de tener una buena estructura para la realización, implementación y mantenimiento del código de lo que será el software.

2050864 CERDA HERRERA JUAN KRISTOF:

Esta actividad permitió ver cómo los requerimientos de la Unidad 2 se convierten en una arquitectura con responsabilidades claras. Separar el dominio, la aplicación y la infraestructura ayudó a identificar dónde debe vivir cada regla: lo que el negocio permite o protege se queda en el dominio, los servicios de aplicación solo coordinan y los repositorios se limitan a guardar y recuperar.

También quedó claro que la arquitectura no se construye de forma lineal. Al contrastar el diseño con el BRD, las User Stories y el BPMN aparecieron reglas que se habían omitido o ubicado en una capa incorrecta, como la autorización de Operaciones para cancelar con unidad asignada o la regla de disponibilidad del operador, y fue necesario corregirlas en lugar de asumirlas. La IA sirvió como apoyo para proponer y

revisar alternativas, pero cada propuesta tuvo que verificarse contra los documentos del proyecto.

Finalmente, las decisiones tomadas tienen costos que se aceptaron de forma consciente, como la mayor complejidad inicial del Domain Model o el mantenimiento de un modelo de lectura separado, a cambio de reglas que se pueden modificar en un solo lugar y de consultas que no dependen del modelo de escritura. Quedan como trabajo posterior algunos puntos que el alcance actual no cubre, como los tiempos por fase, el rechazo de asignación por parte del operador y la alerta a Ventas y Operaciones al cancelar sin unidad.

2066053 ESPARZA TORRES CARLOS ARTURO:

Con esta actividad entendí mejor cómo se separan las responsabilidades de un sistema en dominio, aplicación e infraestructura. Mi parte fue el caso de reasignar una unidad por avería: diseñe el IncidenteRepository, el diagrama de clases, los dos diagramas de secuencia y el ADR sobre por qué el Incidente debe ser su propio Aggregate y no vivir dentro de la Orden de Servicio.

Lo que más me ayudó fue revisar mi propuesta contra el BRD, el glosario y las User Stories. Al hacerlo encontré errores que no había notado: había usado la fase “En ruta” cuando el glosario dice “En tránsito”, había supuesto que el incidente se creaba al reasignar cuando en realidad lo reporta el Operador, y había puesto a Operaciones a cancelar la orden a las 4 horas cuando la historia US-011 dice que lo hace el sistema. También me di cuenta de que una regla de negocio había quedado en el Application Service, así que la moví al dominio.

Aprendí que la IA sirve para proponer y acelerar el trabajo, pero cada resultado hay que verificarlo contra los documentos del negocio. Me quedan pendientes por confirmar con el equipo si la cancelación por falta de reemplazo lleva penalización y que el aviso al cliente incluya los nuevos horarios de entrega.